



Dynamic Risk-based Two-Factor Authentication (2FA) System

Adaptive 2FA that applies OTP or manual review only when contextual risk is detected.

Internship Project — Ashutosh Kumar, IIT Patna

Problem Statement

Passwords and static 2FA (OTP on every login) are widely used but insufficient: they either create friction for legitimate users or fail to stop sophisticated attacks. Static OTPs can be bypassed by phishing, SIM-swap, and credential stuffing, while traditional authentication ignores contextual signals that indicate risk.

Static 2FA applies equal friction to all logins → poor user experience.

Attack vectors that bypass static 2FA: phishing, credential stuffing, SIM-swap.

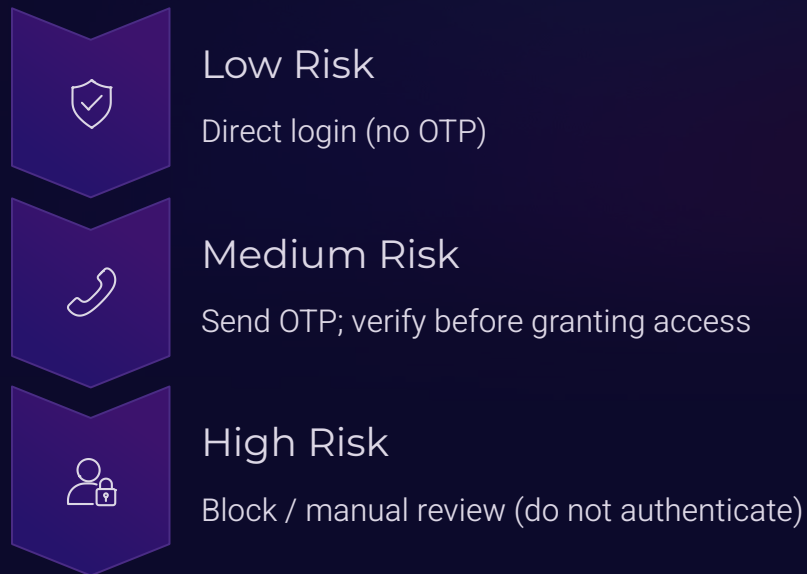
Logins lack contextual checks (device, IP, geolocation, login time, velocity).

Need: an explainable, context-aware system that only challenges risky attempts.

 **Example:** Password correct but login from new country — should be higher risk.

Proposed Solution

Build a rules-based, extensible system that computes a risk score for each login attempt using contextual features (IP, device, geo, time, failed attempts). The system maps the score to an action: allow, require OTP, or flag for manual review. This keeps UX smooth for normal users while increasing protection for suspicious logins.



Benefits: better security, fewer false alarms, easily explainable decisions.



Direct access

OTP verification



System Architecture

The prototype is split into a demo frontend and modular backend services. Frontend collects credentials and metadata; backend computes risk, issues OTPs, and manages a small user store. The risk engine is rules-based and can be replaced with an ML model later.

Frontend (Streamlit)

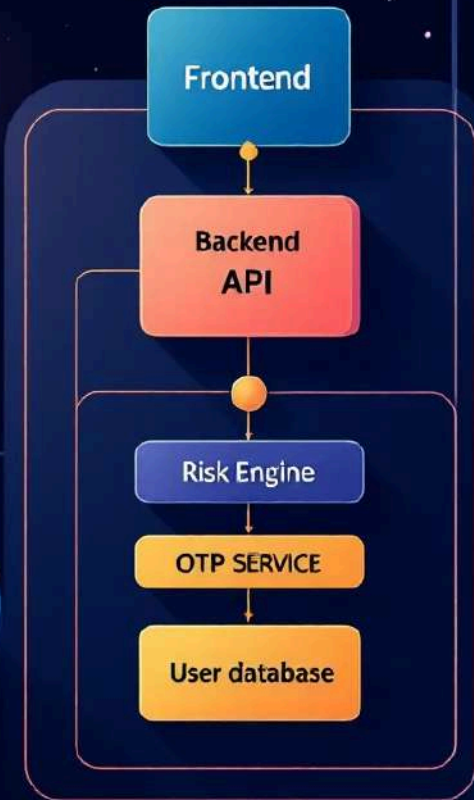
- Login form, OTP input
- Visual display of risk and reasons

Backend (FastAPI)

- Endpoints: /login, /verify_otp, /create_demo_user

Core Components

- Risk Engine: rules-based scoring (weights → score → risk level)
- OTP Service: demo prints OTP to console; optional SMTP/Twilio integration
- User Store: simple JSON file (demo) — upgradeable to SQLite / SQLAlchemy



Risk Features & Scoring

We compute interpretable features from each login attempt and apply simple weighted scoring to determine risk.

Features

- **ip_unknown** — whether the IP is new for the user (binary)
- **geo_distance_km** — haversine distance from last known location (numeric)
- **time_anomaly** — login outside user's usual hours (binary)
- **device_unknown** — new user agent / fingerprint (binary)
- **failed_attempts** — count of failed attempts in recent period (numeric)
- **velocity** — suspicious rapid location changes (optional)

Example Rule Weights

- ip_unknown: **40 points**
- device_unknown: **30 points**
- time_anomaly: **20 points**
- geo_distance_km: **0.5 points per km**
- failed_attempts: **10 points each**

Threshold mapping: < 30 = Low, 30–69 = Medium, >= 70 = High

 **Sample calculation:** If ip_unknown=1 (40) + device_unknown=1 (30) + geo_distance_km=50 (25) => total score = 95 => High risk

Authentication Flow & Examples

User submits username & password plus metadata (IP, UA, lat/lon, timestamp)

Backend computes feature vector from history and request metadata

Risk Engine returns numeric score + human-readable reasons

Backend maps score to action: allow / OTP / manual review

If OTP required, create OTP, send (or print), and wait for verification; if verified → authenticate

Example JSON Responses

✓ Low risk

```
{ "risk_level": "low",  
  "next_action": "success",  
  "explain": {  
    "score": 12.5,  
    "reasons": []  
  }  
}
```

⚠ Medium risk

```
{ "risk_level": "medium",  
  "next_action": "send_otp",  
  "explain": {  
    "score": 45.2,  
    "reasons": [  
      "new IP",  
      "unknown device"  
    ]  
  }  
}
```

✗ High risk

```
{ "risk_level": "high",  
  "next_action": "manual_review",  
  "explain": {  
    "score": 90,  
    "reasons": [  
      "new IP",  
      "time anomaly",  
      "device unknown"  
    ]  
  }  
}
```

Implementation & How to Run

Important Files

- `backend/app.py` – API and endpoints
- `backend/risk_engine.py` – scoring rules & mapping
- `backend/otp_service.py` – OTP generation & console/SMTP send
- `backend/db.py` – demo JSON user store
- `frontend/streamlit_app.py` – demo UI
- `requirements.txt` – dependencies

Quick Setup Commands

create & activate venv

```
python -m venv venv
```

Windows

```
.\venv\Scripts\activate
```

macOS/Linux

```
source venv/bin/activate
```

```
pip install -r requirements.txt
```

start backend

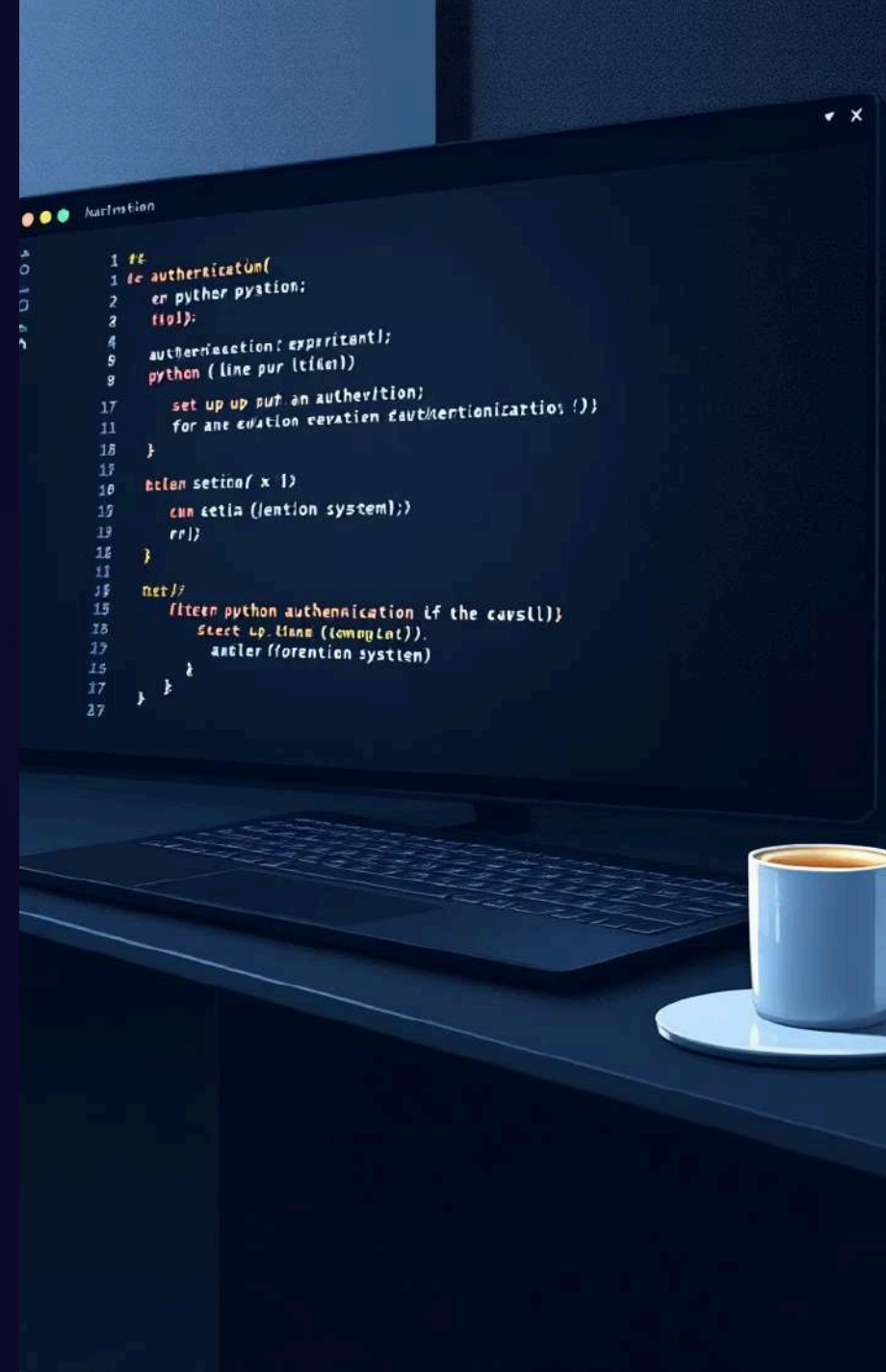
```
cd backend
```

```
uvicorn app:app --reload --port 8000
```

start frontend (new terminal)

```
cd ../frontend
```

```
streamlit run streamlit_app.py
```



How to Demo

Create Demo User

In Streamlit, click Create demo user (alice)

Login

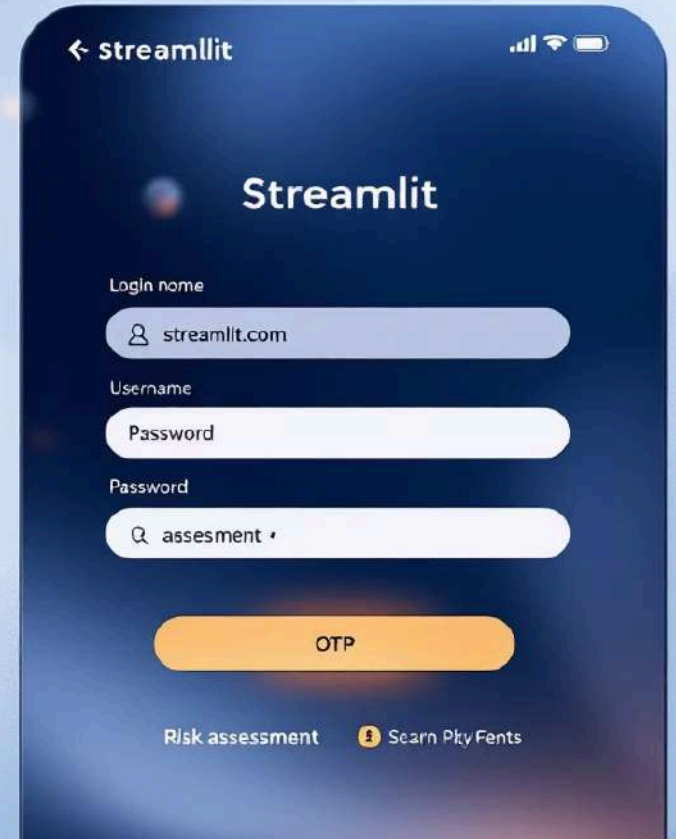
Login with alice / demo-password

Check Response

Observe the returned JSON in the UI; if OTP required, check backend console for printed OTP and enter it in UI

Test Different Scenarios

Repeat changing latitude/UA to generate low/medium/high scenarios



Conclusion & Deliverables

This prototype demonstrates a practical, explainable approach to adaptive authentication: it reduces unnecessary friction for legitimate users while elevating checks for risky logins. The modular design enables rapid extension to ML models and integration with production identity systems.

Deliverables

- Source code: backend/ + frontend/ (GitHub repo link)
- README with run instructions and screenshots
- This slide deck (8 slides) and optional short demo video (30–60s)

Contact

Repo:
<https://github.com/ashutosh8021/dynamic-risk-2fa>

Email:
ashutoshkumarr802152@gmail.com





Future Work

Enhanced Data Storage

Replace JSON store with SQLite/SQLModel and secure password hashing (bcrypt)

Security Improvements

Implement rate limiting on login and OTP endpoints

OTP Delivery

Integrate SMTP/Twilio for real OTP delivery and move OTPs to a secure store (hashed, TTL)

Machine Learning

Train and compare an ML classifier (RandomForest) against rule baseline

Thank You!